ChatGPT

AI-Powered Talent Scouting & Engagement Agent

Real-World Talent Systems

- **LinkedIn Recruiter:** Uses a multi-stage search and ML ranking pipeline. Queries (free-text or structured like titles/skills) are parsed and sent to a Lucene-based index ("Galene" with inverted/forward fields) [1] [2]. A distributed broker fans out the query, retrieves candidates by text match, and applies ML models. Candidates are scored on skill/experience similarity, location, and response likelihood; top candidates are re-ranked with dynamic features (e.g. recent activity) [2]. Crucially, LinkedIn optimizes for *mutual* interest – e.g. measuring InMail Accept (candidate's positive reply) as the main objective [3].
- **Eightfold AI:** Uses deep embedding and rich feature engineering. Each resume and JD is embedded via a pretrained LLM and fine-tuned models into a high-dimensional semantic space [4]. In addition, structured features are extracted (skill overlap, title/seniority fit, company/industry similarity, career trajectory) using token embeddings and RNNs [5] [6]. A custom "Match Score" model combines these semantic and interpretable features (with fairness constraints built in) to rank candidates holistically [7] [5]. Eightfold emphasizes explainability (breaking down matching factors) and bias mitigation in its scoring [8].
- **SeekOut:** Aggregates data and semantic search. It builds a **unified candidate profile** from 1B+ sources (LinkedIn, GitHub, publications, etc.) to capture skills beyond the resume. Their AI stack uses *semantic search* to match accomplishments to job needs even when keywords differ [9]. The system is essentially an "agentic" pipeline: specialized AI agents handle sourcing, filtering, and outreach in an automated workflow [10]. SeekOut touts multi-agent coordination (e.g. one agent parses data, another crafts messages) and advanced search to surface passive talent that keyword tools miss [9].
- **HireVue (Interviewing/Assessment):** Focuses on video and conversational AI. Its platform analyzes recorded or live video answers using deep CV and NLP models to assess communication, confidence, and problem-solving [11] [12]. It also includes a sophisticated **conversational AI** chatbot for candidate interaction (via web, SMS, etc.), dynamically generating interview questions and interpreting responses with NLU [13] [14]. HireVue's models (trained on millions of interviews) score candidates on competencies and use structured frameworks to reduce bias [14] [15]. While HireVue is interview-centric, its conversational techniques (adaptive questioning, intent analysis) are instructive for engaging candidates.

Tech Stack Recommendations

- **Language/Framework:** Python is recommended for rapid prototyping (with Flask or FastAPI for APIs) due to rich ML libraries. Node.js (Express) is also viable. For LLM orchestration or multi-agent patterns, frameworks like LangChain, AutoGen, or CrewAI can simplify prompt management and tool use [16] [17].
- **AI Models:** Use a powerful pretrained LLM (e.g. GPT-4, Claude 3) for conversation and text parsing. For embeddings, models like OpenAI's text-embedding-3 (or open LLMs like Llama2/Claude-instant) provide semantic vectors for JDs and resumes [4] [18]. For open-source, Hugging Face's Sentence-Transformers or Cohere's embeddings are options.

- **Databases:** A vector store is key. Options include Pinecone, Qdrant, Weaviate, Milvus or Elasticsearch/OpenSearch with a k-NN plugin [19]. Use a document store (MongoDB, PostgreSQL) or cloud DB (Firebase, Supabase) to hold candidate metadata. The vector DB holds fixed-size embedding vectors for fast similarity search.
- **Frontend:** A simple UI (React, Vue or Streamlit/Dash) to input JD and display results. This can be minimal – even a CLI or notebook output for hackathon.
- **Tools:** PDF/Word parsing libs (PyPDF2, python-docx) for input; RASA or GPT-4's chat API for chatbots; langchain-autogen for agent orchestration if needed [16].

Job Description Parsing

Use modern NLP and LLM techniques to extract structure from a free-text JD:

- **Entity extraction:** Run NER or custom regex to pull out skills, qualifications, titles, experience levels, education, etc. (e.g. spaCy or HuggingFace Transformers fine-tuned on resumes) [18].
- **Transformer models:** Feed the JD through a model like BERT/SBERT or instruct an LLM to list key requirements. For example, prompt GPT-4: *"Extract required skills and years of experience from the following job posting…"*. Contextual embeddings capture synonyms (e.g. "computer vision" vs "CV") [18] [20].
- **Keyword/phrase techniques:** Compute TF-IDF or TextRank to score important terms. Combine this with semantic vectors to not miss synonyms.
- **Data cleaning:** Normalize company and title names via a canonical list to improve matching (as LinkedIn does with "canonical skills") [1].
- **Libraries:** Tools like spaCy, Duckling, or LayoutLM (for PDF layouts) can help transform unstructured JDs into structured fields (skills list, required degree, etc.) [18]. Custom parsers (or LLM-based tools) can handle docs, treating filename or section headers as hints if needed [21].

Candidate Matching & Ranking

- **Embedding & Vector Search:** Pre-compute an embedding for each candidate profile (resume text) and for the job. At runtime, encode the job description into a vector and perform a k-NN search in the vector DB to retrieve top-N candidates whose embeddings are closest (cosine distance) [4] [19]. This captures semantic similarity beyond keywords.

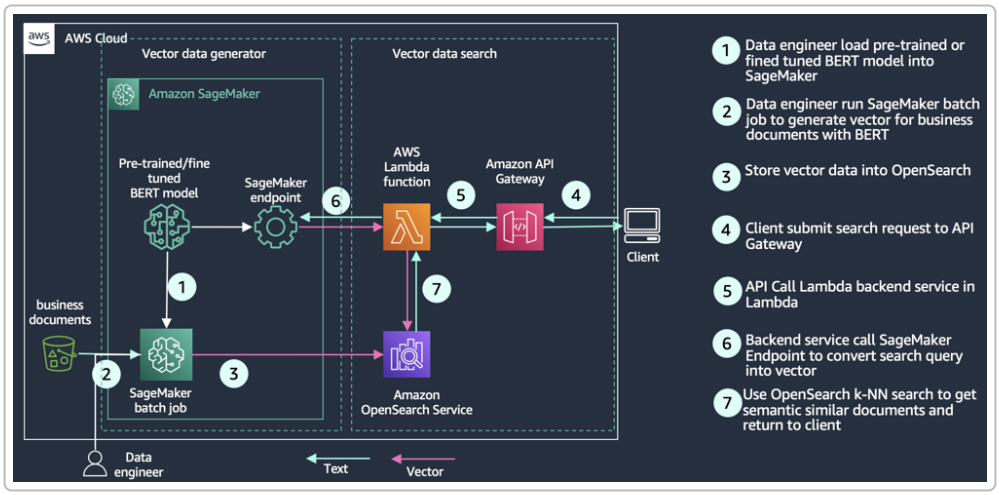

Figure: Semantic search pipeline: job and candidate texts are encoded via a language model (e.g. fine-tuned BERT) into vectors stored in a vector database. At query time, the JD vector is sent to an API (Lambda/Gateway) that performs k-NN search in OpenSearch (or Pinecone) to retrieve the most similar candidate profiles [19].

- **Two-stage ranking:** Use a coarse-to-fine approach. First, retrieve ~100 candidates by fast vector search. Then apply a more expensive re-ranking: either run a cross-encoder (join JD and resume in one model, as a neural ranking step) or compute extra features for each candidate and score them with a trained ML model [22]. For example, Cohere's `rerank` API takes two texts and returns a similarity score [22].
- **Feature signals:** Supplement pure semantic match with explainable signals. For instance, compute *skill overlap score* (cosine similarity of skill-set vectors), *title fit* (compare job title vs candidate's current title), *seniority progression* (predict next title via an RNN and compare to JD's title) [5] [6]. Also consider *industry/company alignment* (vector similarity of past employers to target company) and *recency* (favor skills used recently) [5] [23]. These engineered features make the score interpretable.
- **Ranking logic:** Combine signals in a scoring formula. One example weighting is: `Score = 0.6*(semantic similarity) + 0.4*(structured features)`, where the structured part itself may weight specialty match, seniority, etc. [24]. (In [17], "specialty" and "work style" embeddings were combined with 0.7/0.3, then blended with semantic.) You can adjust weights based on importance.
- **Filtering:** Remove duplicates or stale results. E.g. track which candidates were recently shown (cooldown), and drop those from re-query results [25].
- **Tools:** Many teams use FAISS or Annoy for vector search if on-premise, or cloud services (Pinecone, Milvus) for ease. ElasticSearch/OpenSearch now supports k-NN indices if you already use it [19].

Conversational Candidate Engagement

- **LLM chat agent:** Use a chat model (GPT-4/GPT-3.5 or Claude) to simulate an interactive recruiter. Structure the prompt with system instructions (persona as recruiter) and include context (job details, candidate's profile summary). For example: "You are a hiring manager for *Job X*. Engage the candidate by asking relevant questions (experience, interest, next steps) and capture their answers." Keep conversation state by feeding all previous Q&A in each turn.
- **Multi-turn flow:** Design a conversation plan: greet candidate, ask about interest level, ask key qualifying questions (e.g. "Do you have experience with [skill]?"), respond to candidate's replies (clarify or delve deeper), and finally conclude (e.g. "Are you interested in learning more about this role?"). Use conditionals in prompts or separate agent instructions for follow-up questions.
- **Prompt engineering:** Ensure clarity by telling the model exactly what not to do (e.g. don't make unsupported promises) and what to do. Templates or chains (via LangChain/AutoGen) can manage multi-turn logic.
- **Interest scoring:** At the end of the chat or after certain questions, derive an "interest score." For simplicity, you can add a final question like "On a scale of 1–5, how interested are you in this role?" and parse the answer. Alternatively, run sentiment analysis or classification on the responses. LinkedIn uses actual responses (InMail accept rate) as interest [3]; we can analogously treat enthusiastic or positive replies as high interest.
- **Tools:** Besides raw LLM calls, frameworks like Rasa or Botpress can handle conversation state, but for a hackathon, simply looping GPT API calls with stored context is fine. The HireVue analysis suggests using NLU to interpret intent and adjust questions dynamically [13]. For example, if the

candidate seems uncertain, the agent might provide more information or ask if they have any concerns.

Scoring & Explainability

- **Match Score:** As above, combine semantic similarity with explicit factors (e.g. skill match, title fit, experience years). Weight these thoughtfully. Example: 60% embedding similarity + 40% structured score [24]. Use domain knowledge to set weights (e.g. put more weight on critical skills, less on optional criteria).
- **Interest Score:** Derive a numerical score from the conversation. This could be a weighted combination of self-reported interest and sentiment (e.g. positive language = higher). You might simply average the semantic answer for "how interested" and a NLP-based confidence measure. Keep it on a comparable scale (e.g. 0–1).
- **Final Ranking:** Combine Match and Interest (e.g. `Total = α·MatchScore + β·InterestScore`) to sort candidates. Often, match gets higher weight for relevance but interest filters out passive candidates.
- **Explainability:** Show why each candidate was ranked as they are. List key matching points (e.g. "Matches 4 of 5 required skills: Java, Kubernetes, AWS; title progression to Senior Developer"). This mirrors smart hiring pipelines that highlight contributing factors [26]. For instance, output something like: "Tech match: 0.82 (strong), Experience level: 5 years (meets); Interest: High." Providing a short explanation (matched skills, expected next title) helps recruiters trust the system.
- **Bias & Fairness:** Incorporate checks to prevent demographic bias. Use standardized features (skills, years) rather than personal attributes. Eightfold emphasizes fairness by design, and LinkedIn has deployed "gender-representative ranking" for diversity [8]. At prototype stage, ensure you do not hard-code any biased rules (like excluding age or graduation date from scoring).
- **Metrics:** Evaluate with proxies like precision@k (how many top-k candidates actually respond positively) or InMail-Accept-like measures [3]. In a hackathon, you can manually inspect a few matches or do a quick user test to validate logic.

Architecture & Implementation Steps

A practical prototype can follow this pipeline:
1. **Input & Parsing:** User uploads or pastes the JD. The system extracts structured criteria (skills, title, location) via NLP or prompt.
2. **Index Candidates:** Pre-load your candidate dataset. For each profile (resume), preprocess text (clean, lowercase) and compute an embedding vector (e.g. via an LLM or SentenceTransformer). Store vectors in the vector DB; keep original profile data in a secondary store.
3. **Retrieve Matches:** When a JD arrives, encode it into an embedding and query the vector DB for top ~100 nearest candidates (using cosine similarity).
4. **Compute Scores:** For each retrieved candidate, calculate detailed scores. Combine the vector-similarity score with engineered features: count of overlapping skills, normalized experience gap, title match, etc. [5] [24]. Sum them with chosen weights to get a Match Score (0–1).
5. **Shortlist Candidates:** Filter out very low scores (< threshold) and rank the rest by Match Score.
6. **Engage Candidates:** For the top N candidates (or until resources), invoke a conversational agent. Feed the candidate's details and role context into an LLM chat. The agent simulates asking questions and records replies.
7. **Interest Scoring:** Analyze each conversation. If the candidate explicitly says interest level, use it;

otherwise, run a quick sentiment/intention classifier on their final statements. Assign an Interest Score (e.g. 0–1).

8. **Final Ranking & Explanation:** Compute combined `FinalScore = w1·Match + w2·Interest`. Sort candidates by FinalScore. Output a table showing name, Match Score, Interest Score, and a bullet-point explanation (e.g. "Skills match: Java, AWS; Intended role level: Senior; Interest: Enthusiastic").

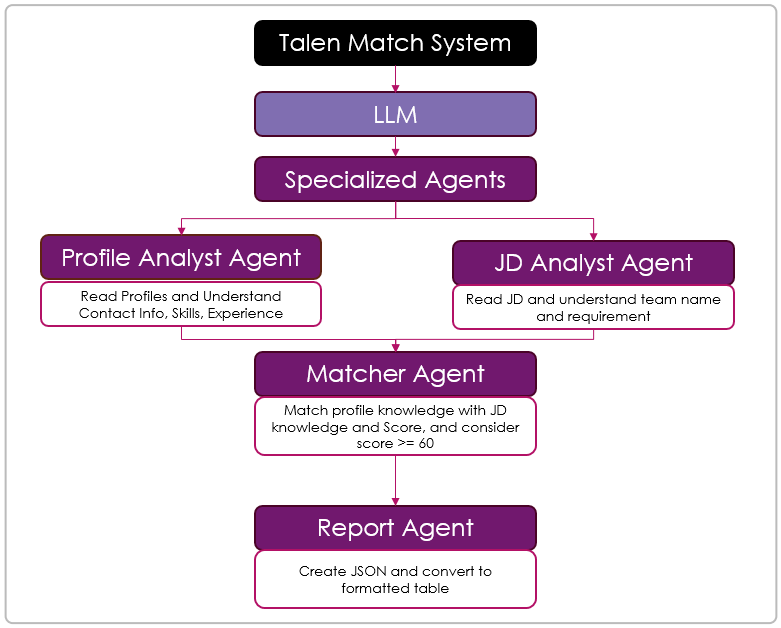

Figure: *Multi-agent matching architecture:* an LLM orchestrates specialized agents. The "Profile Analyst" extracts info from resumes, the "JD Analyst" parses the job description, the "Matcher" computes scores, and the "Report" agent formats the output. This modular design (inspired by recent implementations) allows clear separation of tasks (here illustrated schematically).

Trade-offs and Best Practices

- **Simplicity vs. Sophistication:** For a hackathon, favor off-the-shelf APIs (e.g. OpenAI for embedding and chat) to demonstrate features quickly. This is simpler but incurs API costs and latency. Using open-source models (Llama2, Falcon) avoids cost but needs local compute.
- **Multi-Agent vs. Monolith:** A true multi-agent system (like CrewAI) is elegant but adds complexity. A single pipeline with well-structured functions may be faster to build. You can simulate agents by sequentially calling different functions or subprocesses. The multi-agent approach (Fig. above) is impressive but optional.
- **Performance vs. Accuracy:** Embedding search is fast but imperfect. Adding a reranker (cross-encoder) improves accuracy at the expense of speed. For a prototype, retrieving ~50 candidates and reranking them offline (or asynchronously) is acceptable.
- **Real-time vs. Batch:** Real-time updating of candidate data (new resumes, status changes) makes the system more realistic (LinkedIn emphasizes index freshness [27]), but you can start with a static sample set. Document and update assumptions clearly.
- **Explainability:** Always show how a score was derived. Black-box match scores are hard to trust. List matched skills or the most influential features (like "skill overlap 0.9" or "predicted next title: Senior Dev"). Tools like SHAP are overkill here; simple rule-based breakdown suffices [26].

- **Data Quality:** Ensure resumes and JDs are well-formatted. In practice, cleansing (removing PDF noise, correcting OCR) improves extraction accuracy. For hackathon data, include a few clean examples to show end-to-end workflow.
- **Bias/Audit:** Even in a demo, avoid encoding biased criteria. If available, show that the system does not use attributes like gender. Eightfold and LinkedIn have bias-mitigation steps [8] [3]; at minimum, note this concern in presentation.
- **Human-in-the-loop:** For high-impact matches, consider having a human verify before contacting a candidate. In [17], every automated match was reviewed by a human analyst, improving trust [28]. You can mention that as a best practice, though not required to implement now.
- **Documentation:** Comment and log what each component does. This is crucial in a demo so judges can understand your flow. Use structured JSON for intermediate results so front-end can easily display scores and reasons.
- **Limit Scope:** Finally, prioritize a minimal viable flow: JD → parse → embed search → simple scoring → LLM chat → result. It's better to have a working end-to-end demo (even with limited data) than unfinished advanced features.

Each component above is grounded in real systems (see citations) and represents a balance of practical simplicity and demonstrable power. For example, LinkedIn's system inspired the idea of layered retrieval and mutual-interest metrics [3] [2], while Eightfold showed the value of embeddings plus structured features [5]. By combining these ideas in a clear pipeline (as sketched in Figure and steps), you get a prototype that is simple enough for a hackathon yet rich enough to impress.

Sources: Industry engineering blogs and papers on recruitment tech [1] [7] [5] [29] [13], plus practical implementation accounts [22] [16]. These informed the design choices, algorithmic details, and best practices above. Each recommendation is based on real-world examples of talent-matching systems.

---

[1] [2] [3] [27] AI Behind LinkedIn Recruiter Search and Recommendation Systems
https://www.linkedin.com/blog/engineering/recommendations/ai-behind-linkedin-recruiter-search-and-recommendation-systems

[4] [5] [6] [7] [8] [23] AI-powered talent matching: The tech behind smarter hiring
https://eightfold.ai/engineering-blog/ai-powered-talent-matching-the-tech-behind-smarter-and-fairer-hiring/

[9] [10] [29] Are SeekOut's AI Agents Going to Redefine the Recruiter's Role - Brandon Hall Group
https://brandonhall.com/are-seekouts-ai-agents-going-to-redefine-the-recruiters-role/

[11] [12] [13] [14] [15] HireVue: AI Video Interviewing & Assessment
https://digidai.github.io/2025/07/26/hirevue-deep-analysis/

[16] [17] [21] Building an AI-Powered Talent Match System with Multi-Agent Architecture
https://www.linkedin.com/pulse/building-ai-powered-talent-match-system-multi-agent-ravi-prabakaran-t7aoe

[18] [20] [26] Smart-Hiring: An Explainable end-to-end Pipeline for CV Information Extraction and Job Matching
https://arxiv.org/html/2511.02537v1

[19] Amazon OpenSearch Service's vector database capabilities explained | AWS Big Data Blog
https://aws.amazon.com/blogs/big-data/amazon-opensearch-services-vector-database-capabilities-explained/

22 24 25 28 I Built a Semantic Job Matching Pipeline: Here’s How It Works | by Marco Ferri | Mar, 2026 | Medium
https://medium.com/@marcoferri462/i-built-a-semantic-job-matching-pipeline-heres-how-it-works-403e467e837e